PRAKTIKUM UTS KECERDASAN BUATAN

Nama : Oktova Yoga Praditia Samosir
NIM : 42324001 (NIM Ganjil)
Prodi : D3 Teknologi Informasi
Mata Kuliah : Kecerdasan Buatan
Tahun Ajaran : Semester Gasal T.A. 2025/2026

Panduan Pengerjaan:

Notebook ini berisi jawaban praktikum UTS Kecerdasan Buatan yang disusun secara terstruktur per langkah pengerjaan (*step-by-step*), meliputi:

- **Soal 1**: Eksplorasi Data pada dataset `users_data_cleaned.pickle` (13 langkah eksplorasi)
- **Soal 2**: Regresi Linear pada dataset `Sales_Data.csv` (Prediksi Nilai Penjualan berdasarkan Biaya Promo)
- **Soal 3**: Klasifikasi K-Nearest Neighbors (K-NN) pada dataset `pima-indians-diabetes_data.csv` (Khusus NIM Ganjil)

Setiap langkah pengerjaan dipisahkan dalam sel kode tersendiri dan disertai penjelasan terpisah untuk menerangkan logika kode serta interpretasi hasil outputnya.

Soal 1: Eksplorasi Data `users_data_cleaned.pickle`

1. Load and preview the data

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

path = "users_data_cleaned.pickle"
df = pd.read_pickle(path)
df.head()
```

Out[1]:

	user_id	number_transactions	total_amount_usd	age	job	marital	educ
0	9231c446-cb16-4b2b-a7f7-ddfc8b25aaf6	3.0	2143.0	58	management	married	te
1	bb92765a-08de-4963-b432-496524b39157	NaN	NaN	44	technician	single	seco
2	573de577-49ef-42b9-83da-d3cfb817b5c1	2.0	2.0	33	entrepreneur	married	seco
3	d6b66b9d-7c8f-4257-a682-e136f640b7e3	NaN	NaN	47	blue-collar	married	
4	fade0b20-7594-4d9a-84cd-c02f79b1b526	1.0	1.0	33	None	single	

Penjelasan:

Pada langkah pertama ini, kita mengimpor pustaka (*library*) utama yang dibutuhkan untuk pengolahan data dan visualisasi, yaitu `pandas`, `numpy`, `matplotlib.pyplot`, dan `seaborn`. File dataset `users_data_cleaned.pickle` dimuat ke dalam DataFrame pandas dengan fungsi `pd.read_pickle()`. Kemudian fungsi `head()` digunakan untuk menampilkan 5 baris pertama data agar kita dapat melihat sekilas struktur tabel dan fitur-fitur yang ada, seperti `age`, `job`, `marital`, `education`, `number_transactions`, dan `total_amount_usd`.

2. Check the shape of the data

In [2]:
```
print("Jumlah baris dan kolom:", df.shape)
```

```
Jumlah baris dan kolom: (45209, 19)
```

Penjelasan:

Perintah `df.shape` digunakan untuk melihat dimensi ukuran dataset. Dari output terlihat `(45209, 19)`, yang artinya dataset ini memiliki 45.209 baris data pelanggan dan 19 kolom variabel/fitur.

3. Check the data types

In [3]:
```
df.dtypes
```

Out[3]:
```
user_id                       object
number_transactions          float64
total_amount_usd             float64
age                            int64
job                           object
marital                       object
education                     object
default                         bool
housing                         bool
loan                            bool
contact                       object
duration                       int64
campaign                       int64
pdays                          int64
previous                       int64
poutcome                      object
term_deposit                    bool
date_joined          datetime64[ns]
device                        object
dtype: object
```

Penjelasan:

Fungsi `df.dtypes` digunakan untuk memeriksa tipe data pada setiap kolom. Terlihat bahwa terdapat kolom bertipe numerik integer (`int64`), bilangan pecahan (`float64`), dan tipe teks/kategori (`object`). Pengecekan tipe data ini sangat penting agar kita tahu kolom mana yang siap diproses secara matematis dan kolom mana yang merupakan data kategorikal.

4. Check for correlations

In [4]:
```
df.corr(numeric_only=True)
```

Out[4]:

	number_transactions	total_amount_usd	age	default	housin
number_transactions	1.000000	-0.163409	-0.008813	0.138838	0.03042
total_amount_usd	-0.163409	1.000000	0.095839	-0.065390	-0.06685
age	-0.008813	0.095839	1.000000	-0.017875	-0.18555
default	0.138838	-0.065390	-0.017875	1.000000	-0.00602
housing	0.030429	-0.066857	-0.185552	-0.006020	1.00000
loan	0.075319	-0.084526	-0.015641	0.077232	0.04134
duration	-0.017220	0.022586	-0.004645	-0.010017	0.00504
campaign	0.026431	-0.017274	0.004767	0.016819	-0.02358
pdays	-0.030751	0.006435	-0.023745	-0.029982	0.12419
previous	-0.023046	0.016952	0.001297	-0.018331	0.03708
term_deposit	-0.053390	0.050785	0.025168	-0.022421	-0.13916

Penjelasan:

Fungsi `df.corr(numeric_only=True)` menghitung matriks korelasi Pearson antar variabel numerik. Nilai koefisien korelasi bergerak di antara -1 hingga +1. Nilai mendekati 1 menandakan korelasi linier positif yang kuat, nilai mendekati -1 menandakan korelasi negatif yang kuat, sedangkan nilai mendekati 0 menandakan tidak adanya hubungan linier yang signifikan antar variabel.

5. Generate summary statistics for numerical data only

In [5]:
```
df.select_dtypes(include='number').describe()
```

Out[5]:

	number_transactions	total_amount_usd	age	duration	campaign
count	35210.000000	35210.000000	45209.000000	45209.000000	45209.000000
mean	3.977052	1369.417751	40.935853	258.153067	2.763897
std	3.814329	3063.412688	10.618653	257.525446	3.098076
min	1.000000	-8019.000000	18.000000	0.000000	1.000000
25%	2.000000	73.000000	33.000000	103.000000	1.000000
50%	3.000000	451.000000	39.000000	180.000000	2.000000
75%	4.000000	1438.000000	48.000000	319.000000	3.000000
max	20.000000	102127.000000	95.000000	4918.000000	63.000000

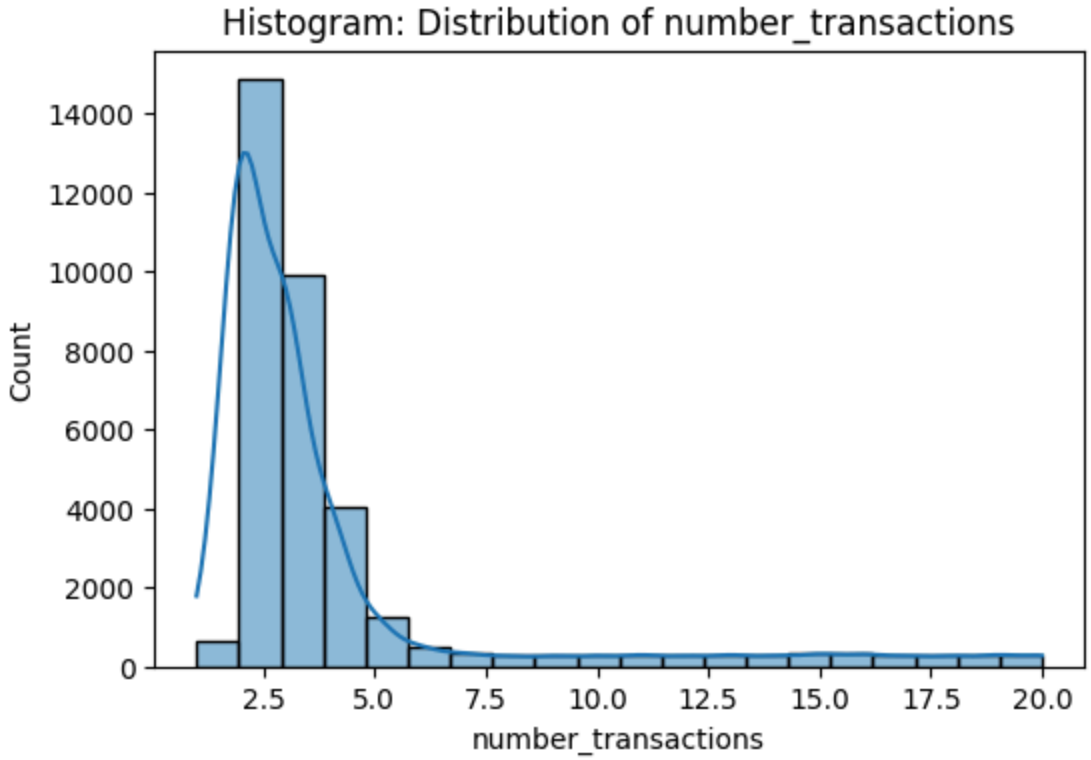
Penjelasan:

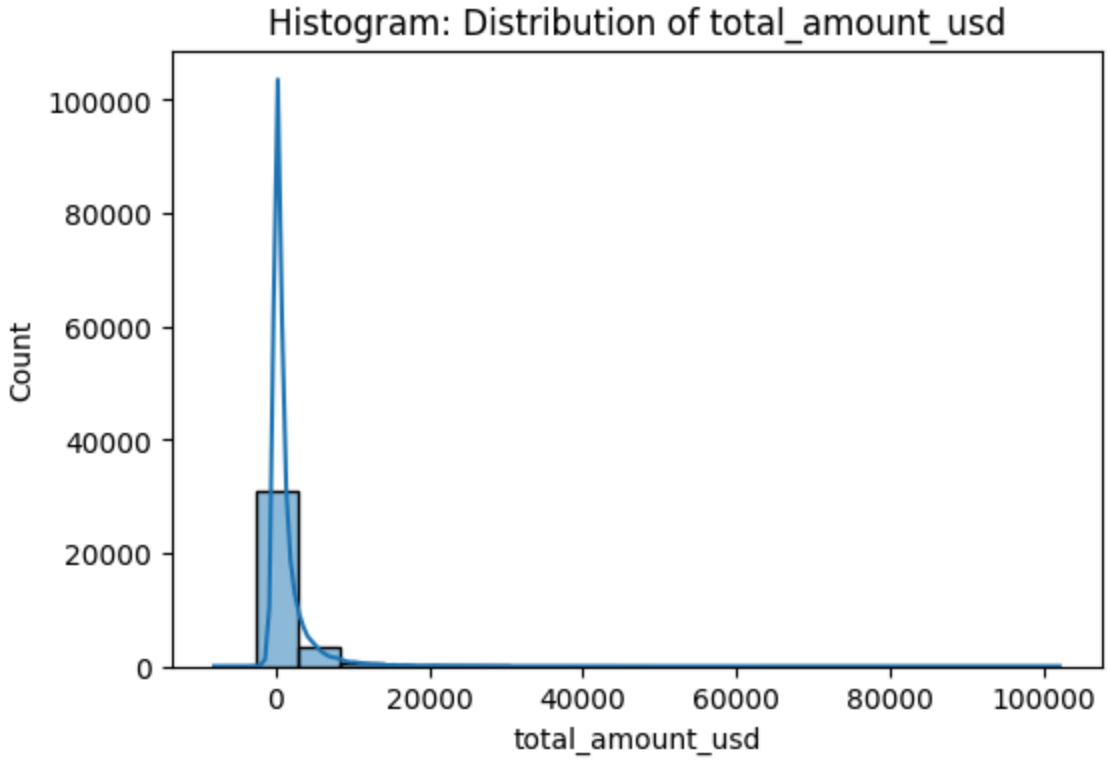
Fungsi `select_dtypes(include='number').describe()` menghasilkan statistik deskriptif khusus untuk seluruh variabel numerik. Informasi yang disajikan mencakup jumlah data (`count`), nilai rata-rata (`mean`), simpangan baku (`std`), nilai minimum (`min`), kuartil bawah 25% (Q1), median/kuartil tengah 50% (Q2), kuartil atas 75% (Q3), dan nilai maksimum (`max`). Dari tabel ini kita dapat mengamati rentang nilai dan mendeteksi sebaran data pengguna.

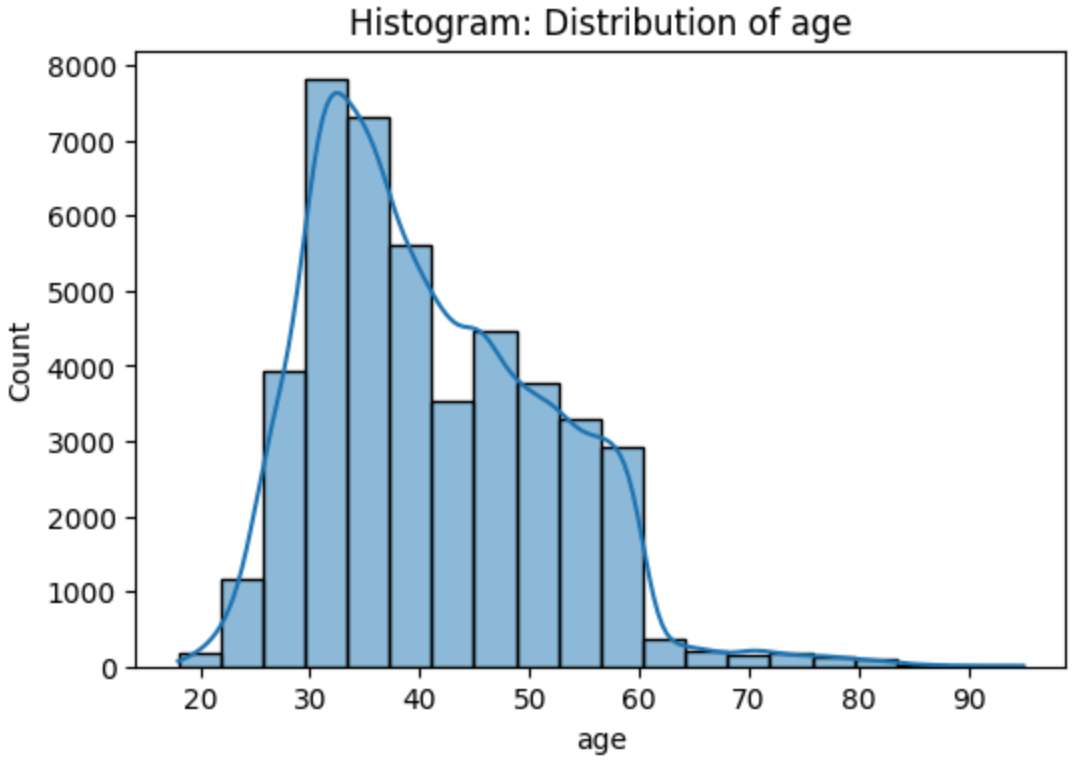
6. Plot Histograms for all numerical columns

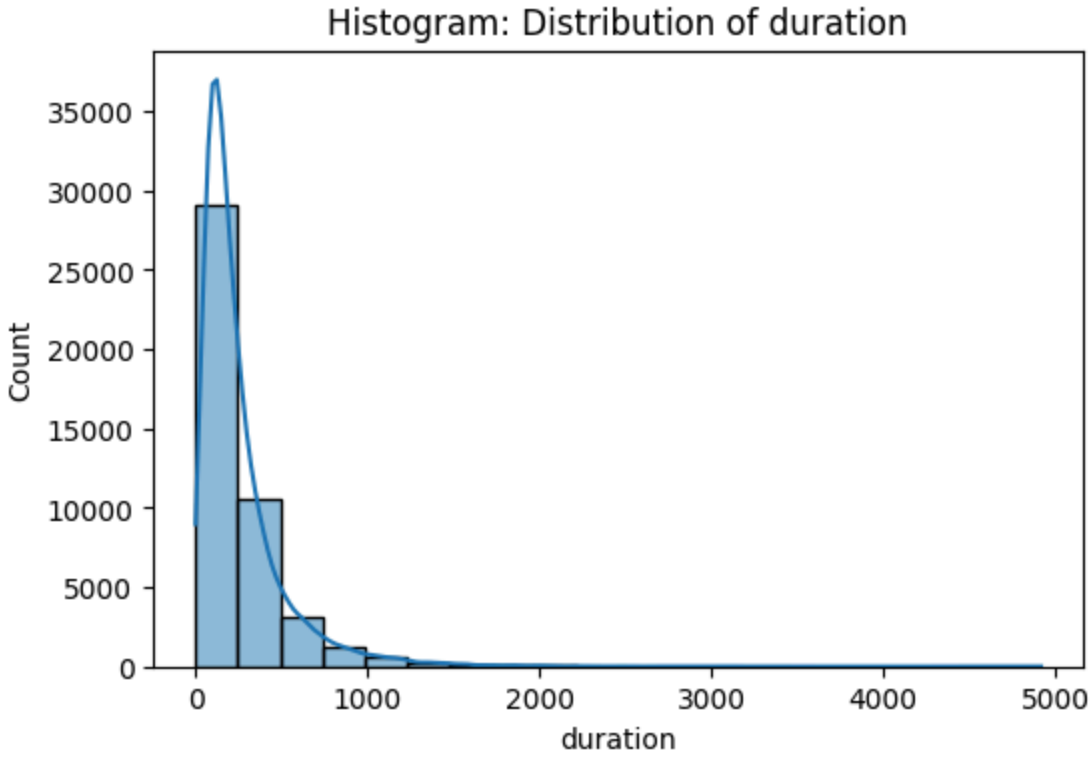
In [6]:
```
numeric_cols = df.select_dtypes(include='number').columns

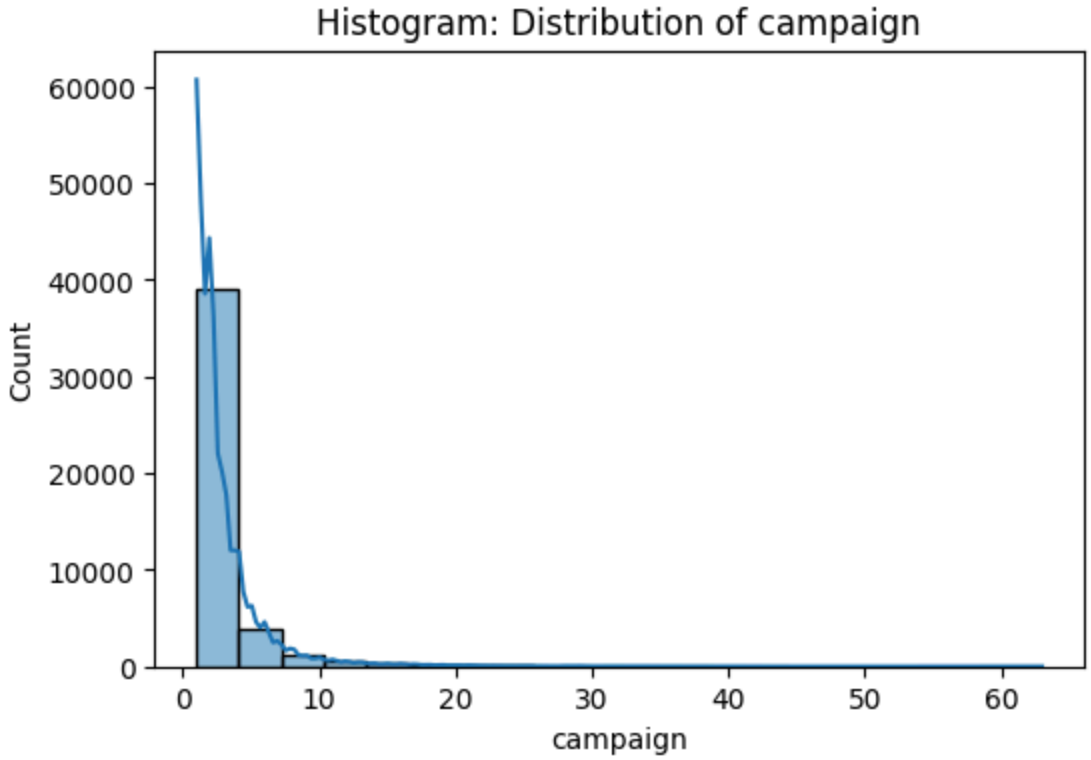
for col in numeric_cols:
    plt.figure(figsize=(6,4))
    sns.histplot(df[col], bins=20, kde=True)
    plt.title(f"Histogram: Distribution of {col}")
    plt.show()
```

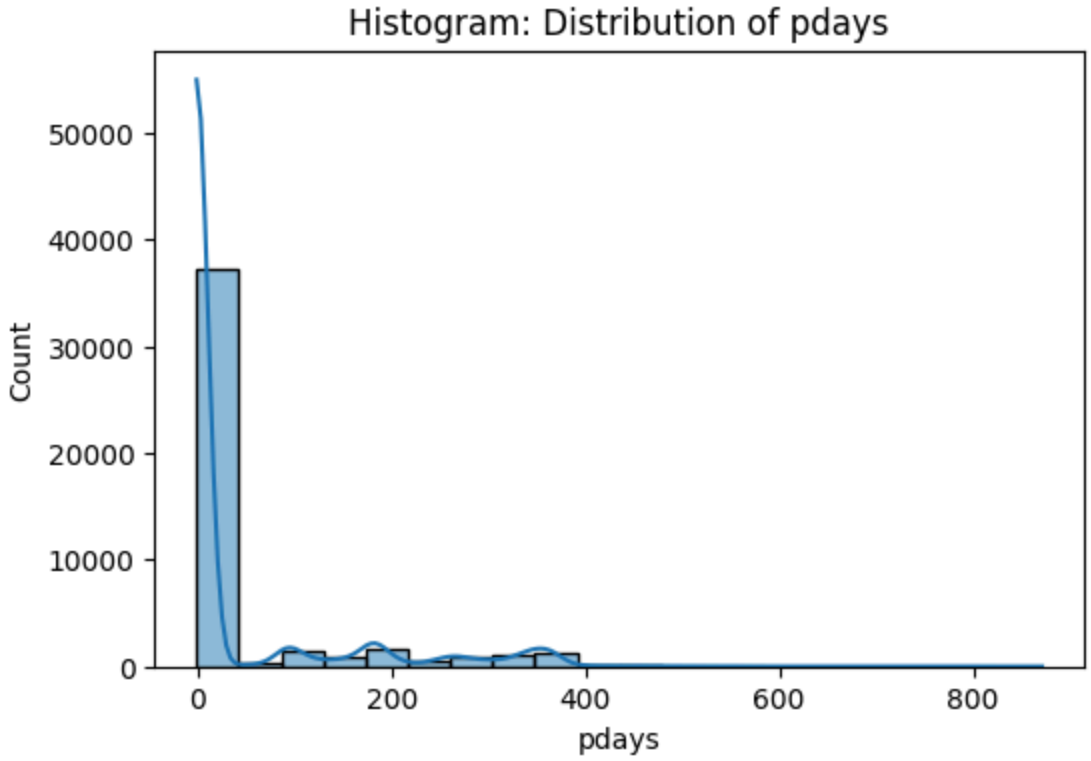

Histogram: Distribution of number_transactions
Count
number_transactions
0
2000
4000
6000
8000
10000
12000
14000
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

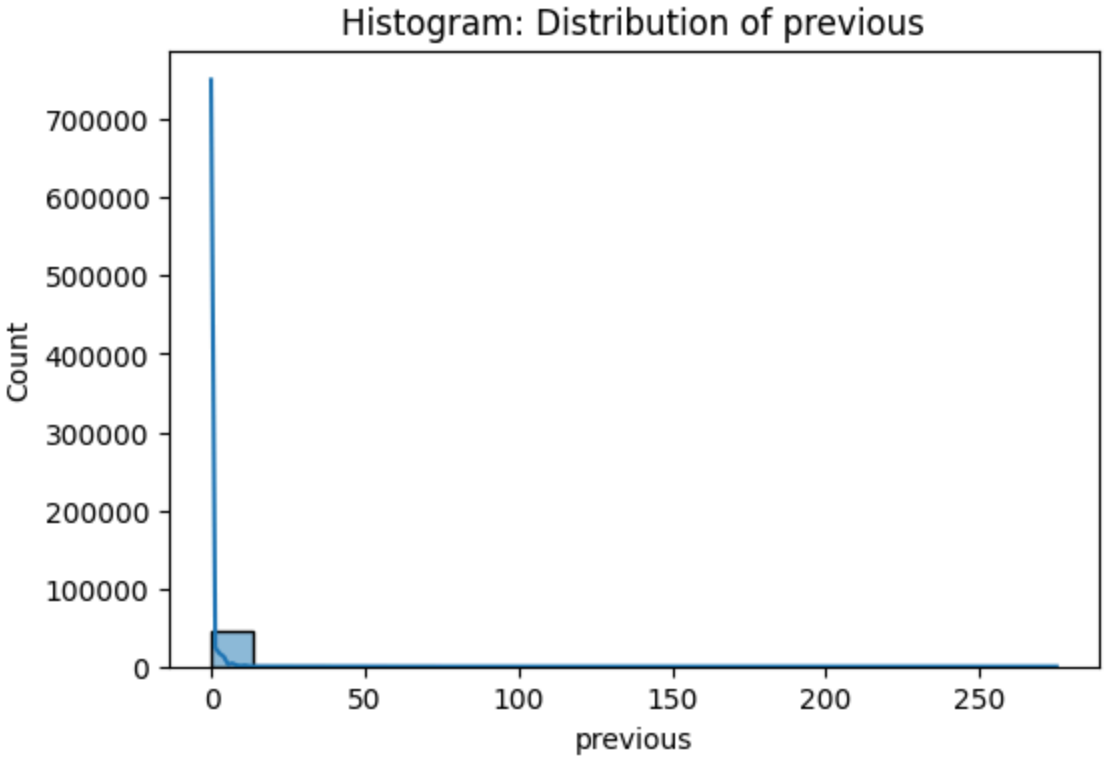

Histogram: Distribution of total_amount_usd
Count
total_amount_usd
0
20000
40000
60000
80000
100000


Histogram: Distribution of age
Count
age
0
1000
2000
3000
4000
5000
6000
7000
8000
20
30
40
50
60
70
80
90


Histogram: Distribution of duration
Count
duration
0
5000
10000
15000
20000
25000
30000
35000
0
1000
2000
3000
4000
5000


Histogram: Distribution of campaign
Count
campaign
0
10000
20000
30000
40000
50000
60000
0
10
20
30
40
50
60


Histogram: Distribution of pdays
Count
pdays
0
10000
20000
30000
40000
50000
0
200
400
600
800



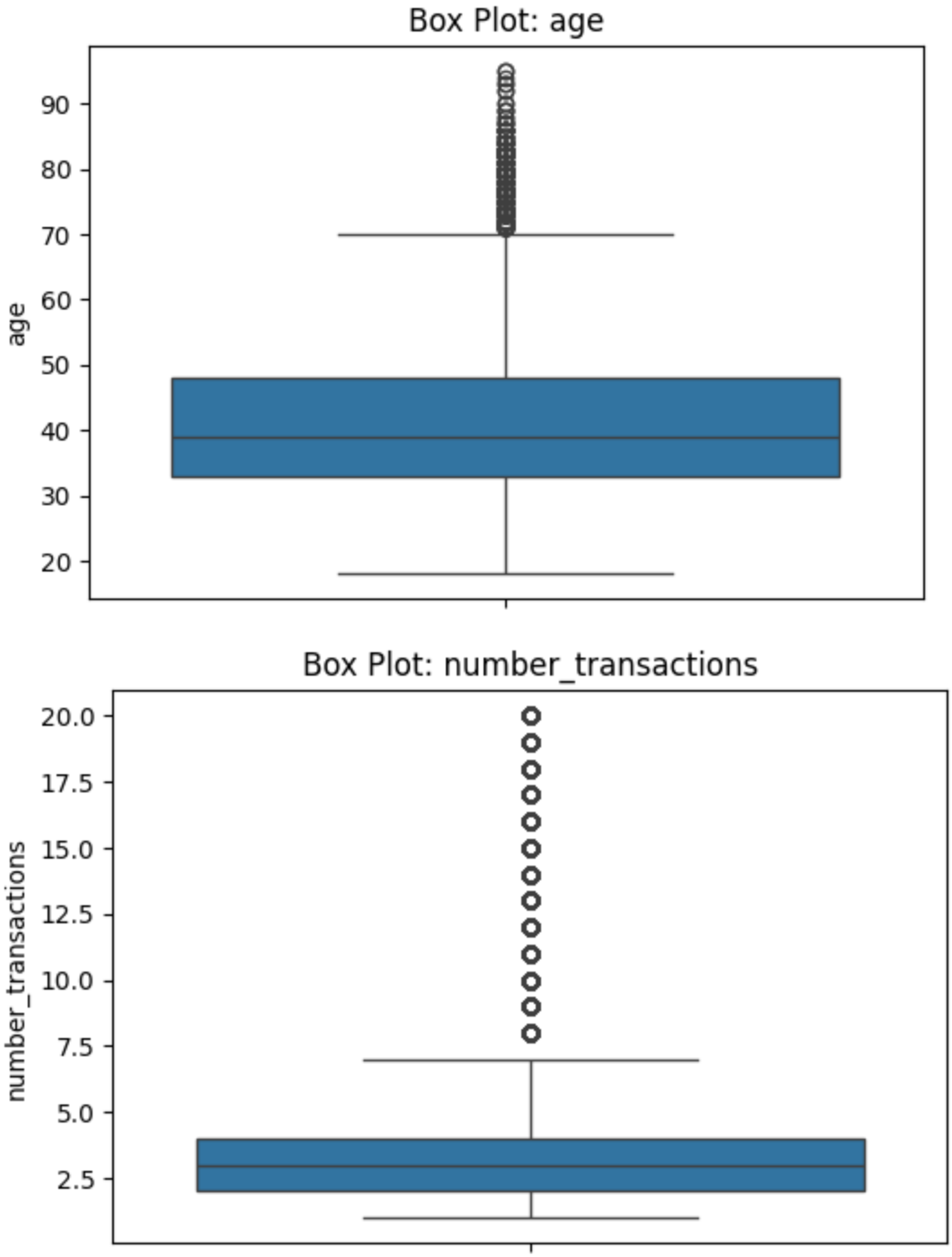
Penjelasan:

Kode di atas melakukan perulangan untuk memplot histogram pada setiap kolom numerik, disertai dengan kurva KDE (*Kernel Density Estimation*). Histogram ini berfungsi untuk melihat bentuk distribusi frekuensi masing-masing variabel, apakah condong (*skewed*) ke kanan/kiri atau simetris menyerupai distribusi normal.

7. Generate a box plot for *age* and *number_transactions*

In [7]:
```
plt.figure(figsize=(6,4))
sns.boxplot(y=df['age'])
plt.title("Box Plot: age")
plt.show()

plt.figure(figsize=(6,4))
sns.boxplot(y=df['number_transactions'])
plt.title("Box Plot: number_transactions")
plt.show()
```



Penjelasan:

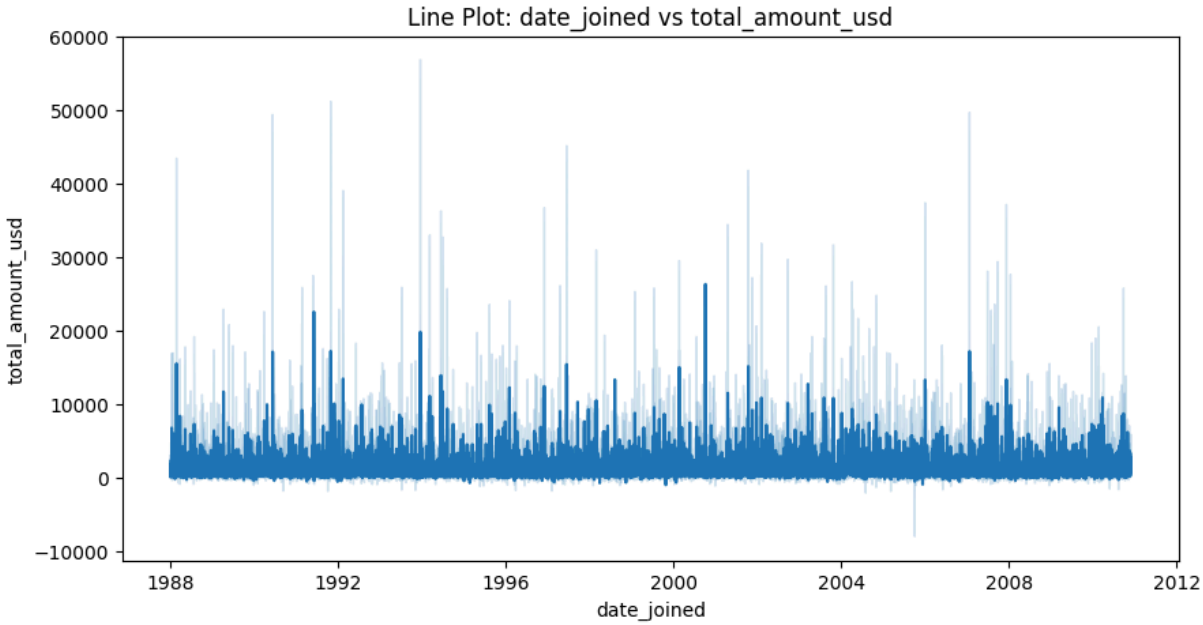
Box plot digunakan untuk memvisualisasikan sebaran data berdasarkan kuartil sekaligus mengidentifikasi titik pencilan (*outlier*).

- Pada variabel `age`, mayoritas usia pengguna berkisar antara 30 hingga 50 tahun, dengan beberapa titik outlier pada rentang usia lanjut (> 70 tahun).
- Pada variabel `number_transactions`, terlihat banyak titik outlier di atas batas atas (*upper whisker*), yang menunjukkan ada sebagian kecil pelanggan dengan frekuensi transaksi yang sangat tinggi dibanding pengguna kebanyakan.

8. Generate a lineplot for *date_joined* to *total_amount_usd*

In [8]:
```
df_sorted = df.sort_values('date_joined')

plt.figure(figsize=(10,5))
sns.lineplot(x='date_joined', y='total_amount_usd', data=df_sorted)
plt.title("Line Plot: date_joined vs total_amount_usd")
plt.xlabel("date_joined")
plt.ylabel("total_amount_usd")
plt.show()
```

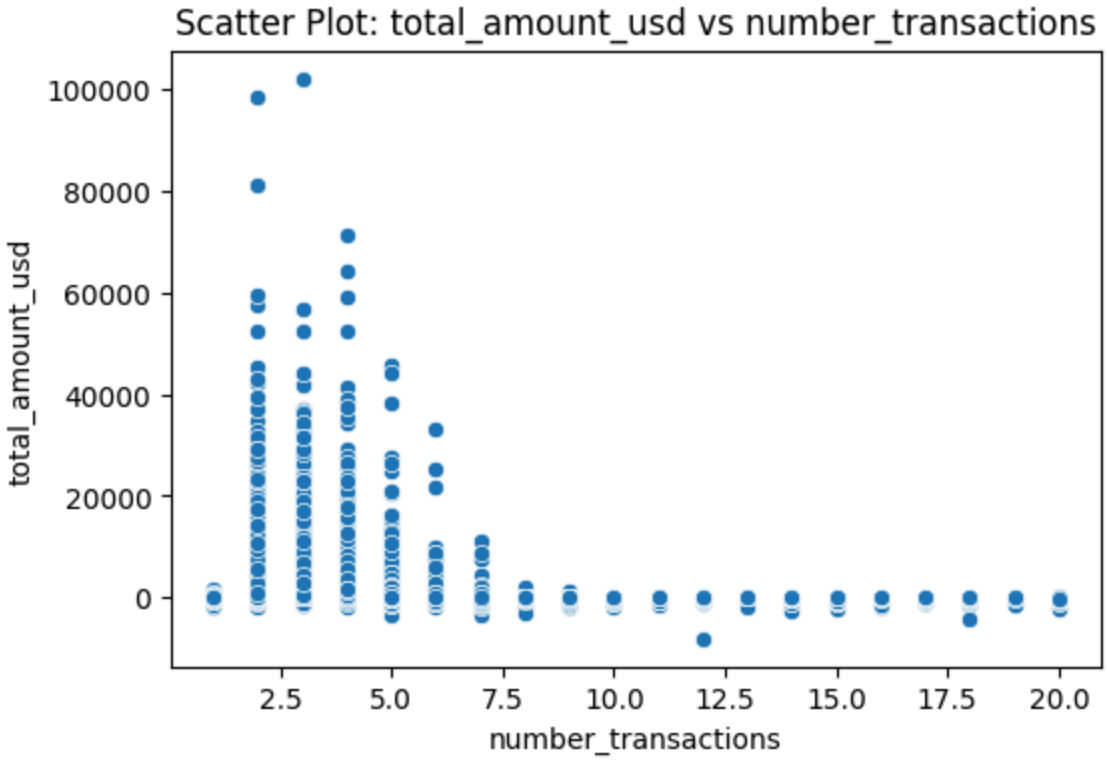


Penjelasan:

Data terlebih dahulu diurutkan berdasarkan tanggal bergabung (`date_joined`) menggunakan `sort_values()` , lalu dibuat grafik garis (*line plot*) untuk melihat pergerakan nilai transaksi pengguna (`total_amount_usd`) seiring berjalannya waktu pendaftaran.

9. Generate scatter plots comparing *total_amount_usd* to *number_transaction*

In [9]:
```
plt.figure(figsize=(6,4))
sns.scatterplot(x='number_transactions', y='total_amount_usd', data=df)
plt.title("Scatter Plot: total_amount_usd vs number_transactions")
plt.show()
```

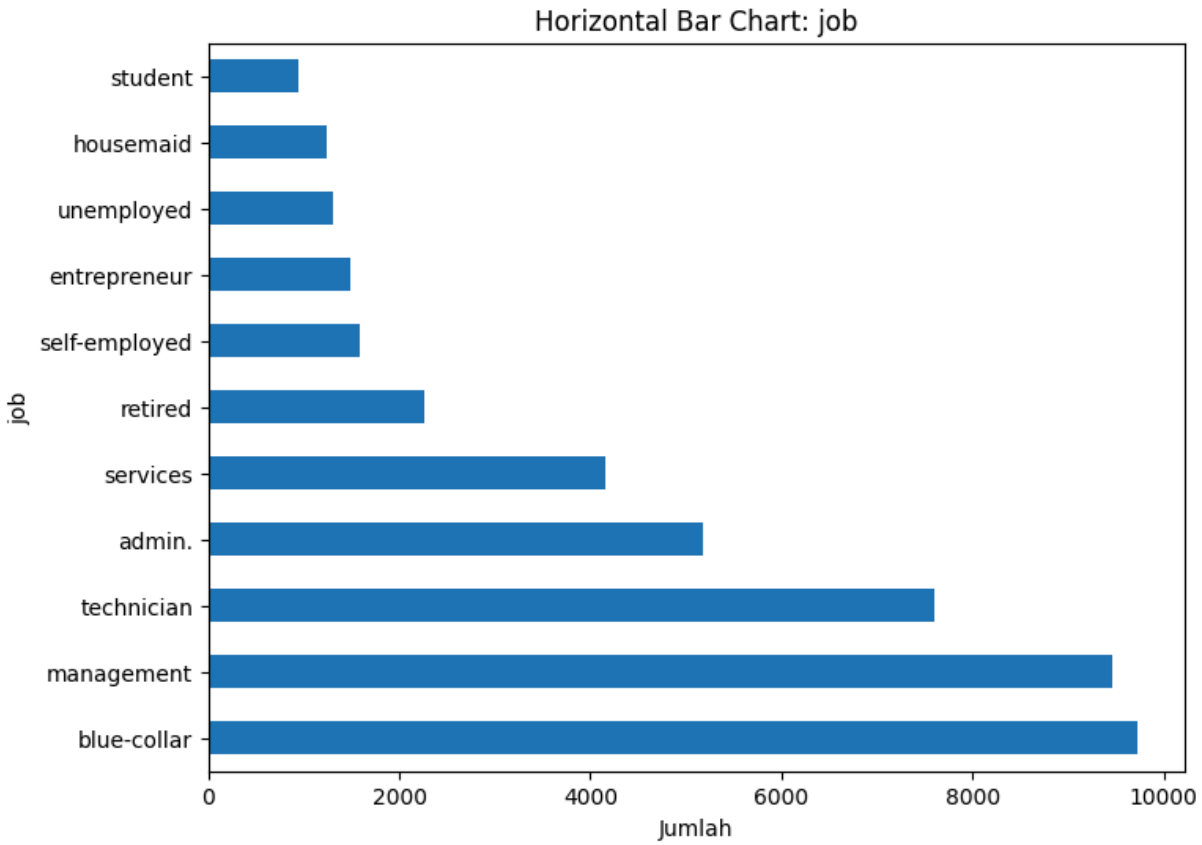


Penjelasan:

Scatter plot ini memetakan korelasi antara jumlah transaksi (`number_transactions` pada sumbu X) dan total uang yang dibelanjakan (`total_amount_usd` pada sumbu Y). Terlihat pola tren positif yang cukup jelas di mana semakin banyak transaksi yang dilakukan pengguna, maka total nominal belanjaannya pun cenderung semakin besar.

10. Generate Horizontal bar charts for *job*

In [10]:
```
plt.figure(figsize=(8,6))
df['job'].value_counts().plot(kind='barh')
plt.title("Horizontal Bar Chart: job")
plt.xlabel("Jumlah")
plt.ylabel("job")
plt.show()
```

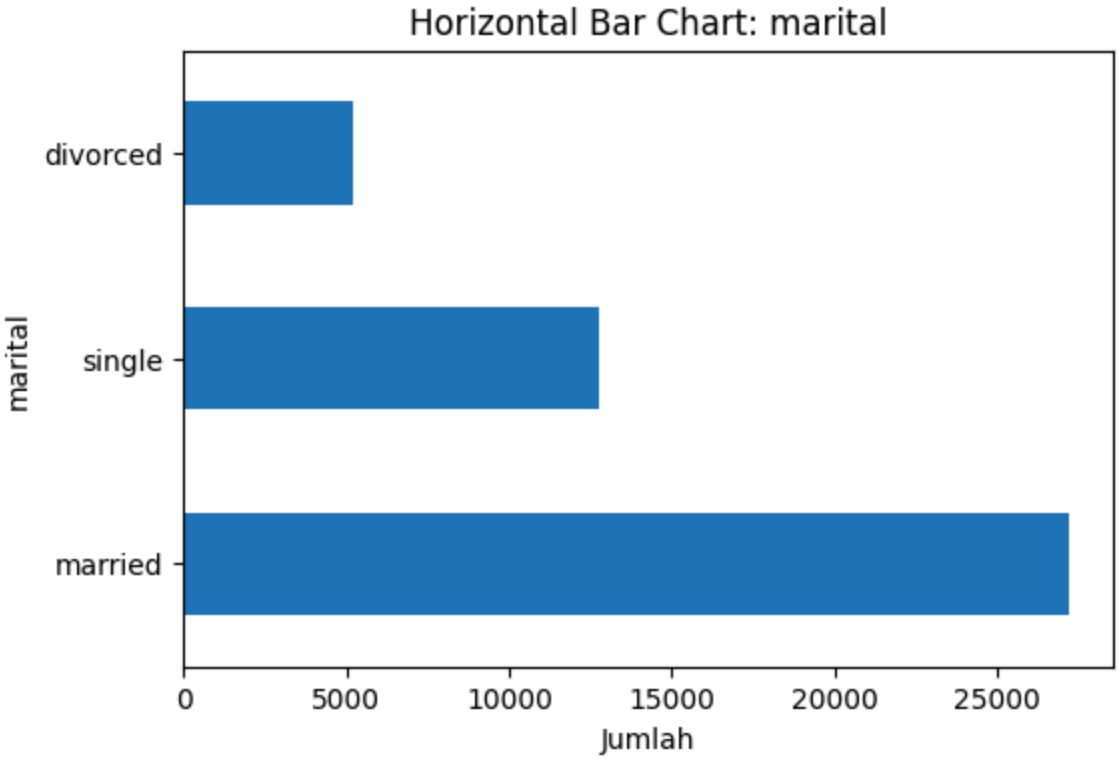


Penjelasan:

Fungsi `value_counts()` menghitung frekuensi setiap jenis profesi pelanggan pada kolom `job`, lalu divisualisasikan dalam diagram batang horizontal (`barh`). Dari grafik terlihat jenis pekerjaan yang mendominasi pelanggan antara lain pekerja kerah biru (*blue-collar*), manajemen, dan teknisi.

11. Generate Horizontal bar charts for *marital*

In [11]:
```
plt.figure(figsize=(6,4))
df['marital'].value_counts().plot(kind='barh')
plt.title("Horizontal Bar Chart: marital")
plt.xlabel("Jumlah")
plt.ylabel("marital")
plt.show()
```

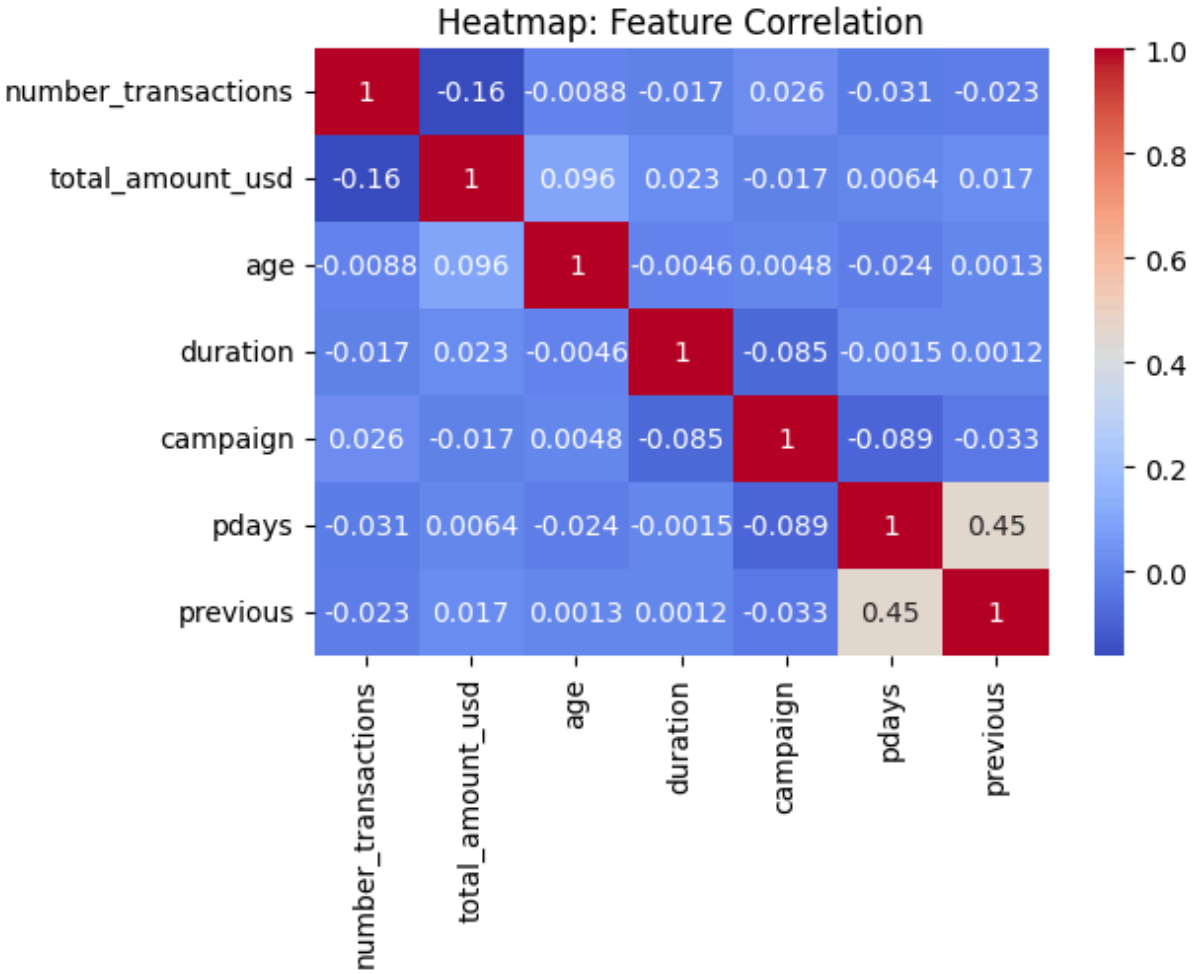


Penjelasan:

Diagram batang horizontal ini menampilkan sebaran status perkawinan (`marital`) pelanggan. Dari visualisasi terlihat mayoritas pelanggan sudah menikah (*married*), disusul pelanggan yang belum menikah (*single*), dan yang bercerai (*divorced*).

12. Generate a heatmap for feature correlations

In [12]:
```
plt.figure(figsize=(6,4))
sns.heatmap(df.select_dtypes(include='number').corr(), annot=True, cmap='coolwarm')
plt.title("Heatmap: Feature Correlation")
plt.show()
```



Penjelasan:

Heatmap korelasi ini memvisualisasikan kekuatan hubungan antar variabel numerik menggunakan warna. Parameter `annot=True` menampilkan nilai numerik koefisien korelasi di setiap kotak, dan colormap `coolwarm` memudahkan kita membedakan hubungan positif (nuansa hangat/merah) dan hubungan negatif (nuansa dingin/biru).

13. Identify Missing Values

In [13]:
```
df.isnull().sum()
```

Out[13]:
```
user_id                  0
number_transactions     9999
total_amount_usd        9999
age                        0
job                      288
marital                    0
education               1857
default                    0
housing                    0
loan                       0
contact                13018
duration                   0
campaign                   0
pdays                      0
previous                   0
poutcome               36957
term_deposit               0
date_joined               30
device                    94
dtype: int64
```

Penjelasan:

Perintah `df.isnull().sum()` memeriksa dan menjumlahkan seluruh data kosong (*missing value*) pada masing-masing kolom. Output ini memberikan gambaran langsung kolom mana saja yang memiliki nilai kosong dalam dataset.

In [14]:
```
percent_missing = 100*df.isnull().mean()
percent_missing
```

Out[14]:
```
user_id                 0.000000
number_transactions    22.117278
total_amount_usd       22.117278
age                     0.000000
job                     0.637041
marital                 0.000000
education               4.107589
default                 0.000000
housing                 0.000000
loan                    0.000000
contact                28.795151
duration                0.000000
campaign                0.000000
pdays                   0.000000
previous                0.000000
poutcome               81.746997
term_deposit            0.000000
date_joined             0.066358
device                  0.207923
dtype: float64
```

Penjelasan:

Menghitung persentase nilai kosong untuk setiap kolom dengan membagi jumlah missing value terhadap total baris dan dikalikan 100%. Informasi persentase ini penting untuk menentukan apakah data kosong tersebut perlu diimputasi (diisi) atau dihapus jika persentasenya terlalu besar.

Soal 2: Linear Regression `Sales_Data.csv`

Memprediksi **Nilai Penjualan** dari **Biaya Promo**.

1.a Membaca file Sales_Data.csv

In [15]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv('Sales_Data.csv', sep=';')
print(df.head())
```

```
   BiayaPromo  NilaiPenjualan
0        1500           90500
1        1800           89500
2        1900          105000
3        2050          102000
4        2050           90500
```

Penjelasan:

Dataset `Sales_Data.csv` dibaca menggunakan fungsi `pd.read_csv()` dengan parameter `sep=';'` karena separator data pada file CSV tersebut menggunakan titik koma. Fungsi `df.head()` menampilkan 5 baris pertama yang memuat data biaya promosi dan nilai penjualan.

1.b Menampilkan variable dalam dataset

Kolom `BiayaPromo` = X (predictor), kolom `NilaiPenjualan` = y (target).

In [16]:
```
print("Nama kolom:", list(df.columns))

X = df.iloc[:, :-1].values
y = df.iloc[:, 1].values

print("Variabel X (Biaya Promo), 5 data pertama:\n", X[:5])
print("Variabel y (Nilai Penjualan), 5 data pertama:\n", y[:5])
```

```
Nama kolom: ['BiayaPromo', 'NilaiPenjualan']
Variabel X (Biaya Promo), 5 data pertama:
 [[1500]
 [1800]
 [1900]
 [2050]
 [2050]]
Variabel y (Nilai Penjualan), 5 data pertama:
 [ 90500  89500 105000 102000  90500]
```

Penjelasan:

Pada tahap ini, kita memisahkan kolom ke dalam dua variabel:

- **Variabel X (Fitur Prediktor)**: Diambil dari kolom pertama (`BiayaPromo`) menggunakan `.iloc[:, :-1].values` .
- **Variabel y (Target)**: Diambil dari kolom kedua (`NilaiPenjualan`) menggunakan `.iloc[:, 1].values` . Keduanya dikonversi menjadi array numpy agar siap diproses oleh algoritma regresi dari pustaka scikit-learn.

1.c Menampilkan data yang di-split (train: 80% & test: 20%)

In [17]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

print("X_train:\n", X_train)
print("y_train:\n", y_train)
print("X_test:\n", X_test)
print("y_test:\n", y_test)
print("Jumlah data train:", len(X_train), "| Jumlah data test:", len(X_test))
```

```
X_train:
 [[3200]
 [4000]
 [2200]
 [3750]
 [2050]
 [1900]
 [2100]
 [3750]
 [3200]
 [2400]
 [4000]
 [3500]
 [2050]
 [1500]
 [3900]
 [3500]]
y_train:
 [153000 170500 109500 198000  90500 105000 104500 187000 173000 150000
 200500 174500 102000  90500 194500 150000]
X_test:
 [[4100]
 [1800]
 [4500]
 [3050]]
y_test:
 [204500  89500 224500 152000]
Jumlah data train: 16 | Jumlah data test: 4
```

Penjelasan:

Fungsi `train_test_split` dari `sklearn.model_selection` membagi dataset menjadi dua bagian: 80% sebagai data latih (*training set*) untuk melatih model regresi, dan 20% sebagai data uji (*testing set*) untuk menguji performa model. Parameter `random_state=0` digunakan agar pembagian data selalu sama (konsisten) saat dijalankan ulang. Dari total 20 sampel data, 16 sampel digunakan untuk train dan 4 sampel untuk test.

1.d Memanggil LinearRegression()

In [18]:
```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)
```

Out[18]:

▾ LinearRegression ⓘ ?

▸ Parameters

Penjelasan:

Pada langkah ini, kita mengimpor kelas `LinearRegression` dari `sklearn.linear_model`, menginisialisasi objek model `regressor`, lalu melatih model pada data latih menggunakan perintah `regressor.fit(X_train, y_train)`. Model akan mencari garis lurus terbaik (*best fit line*) yang meminimalkan jumlah kuadrat selisih antara data aktual dan garis prediksi.

1.e Menampilkan hasil predict dari data test

In [19]:
```
y_pred = regressor.predict(X_test)
y_pred
```

Out[19]:
```
array([197842.1437866 ,  97374.89156365, 215314.70939059, 151976.65907612])
```

Penjelasan:

Fungsi `regressor.predict(X_test)` menghasilkan nilai estimasi penjualan berdasarkan nilai biaya promosi pada data uji. Hasil prediksi `y_pred` adalah array nilai penjualan yang diestimasikan oleh model machine learning.

1.f Menampilkan Grafik Nilai Penjualan berdasarkan Biaya Promosi

In [20]:
```
plt.scatter(X, y, color='red')
plt.title('Nilai Penjualan berdasarkan Biaya Promo')
plt.xlabel('Biaya Promo')
plt.ylabel('Nilai Penjualan')
plt.show()
```

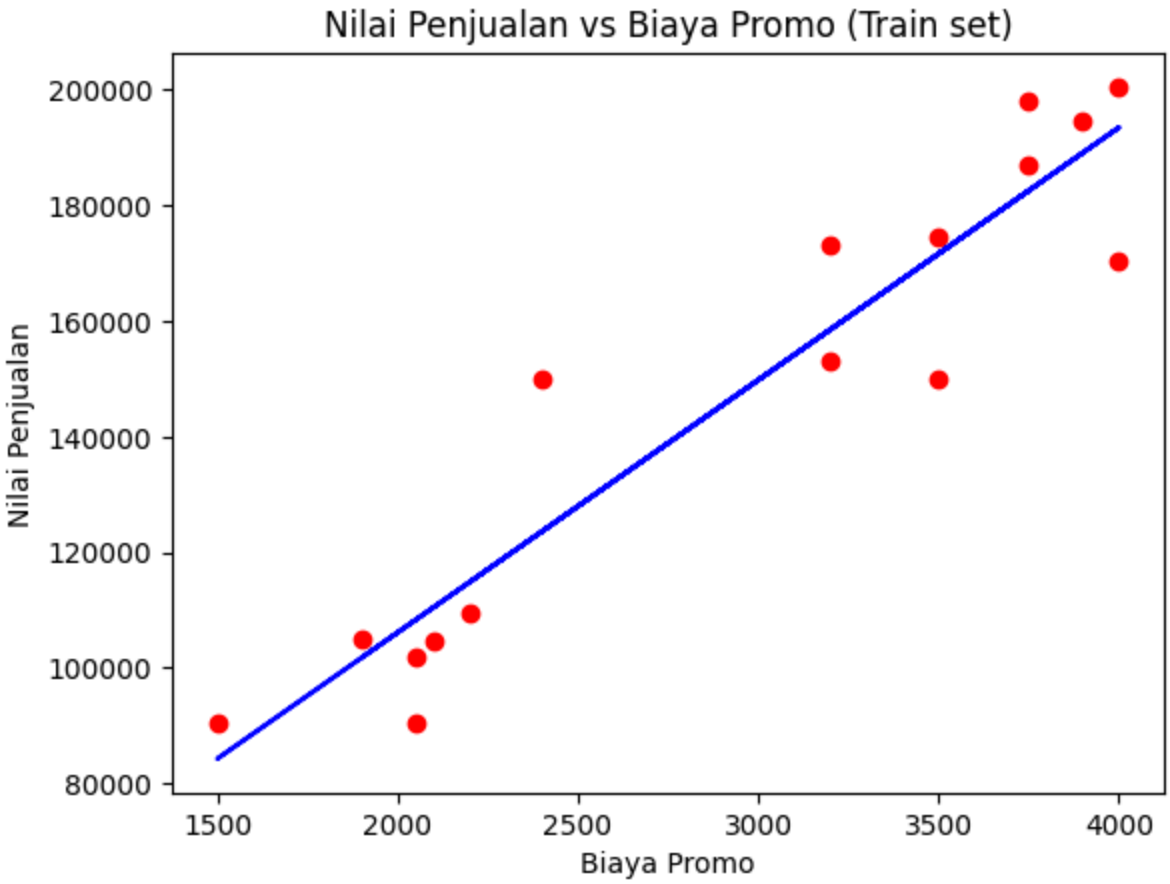


Penjelasan:

Scatter plot dibuat dengan fungsi `plt.scatter()` menggunakan titik-titik berwarna merah untuk memvisualisasikan seluruh data asli antara `Biaya Promo` pada sumbu X dan `Nilai Penjualan` pada sumbu Y. Terlihat jelas adanya korelasi linier positif yang sangat kuat di mana peningkatan biaya promosi diikuti dengan kenaikan nilai penjualan.

1.g Menampilkan garis hasil prediksi pada Biaya Promosi terhadap penjualan (train set dan test set)

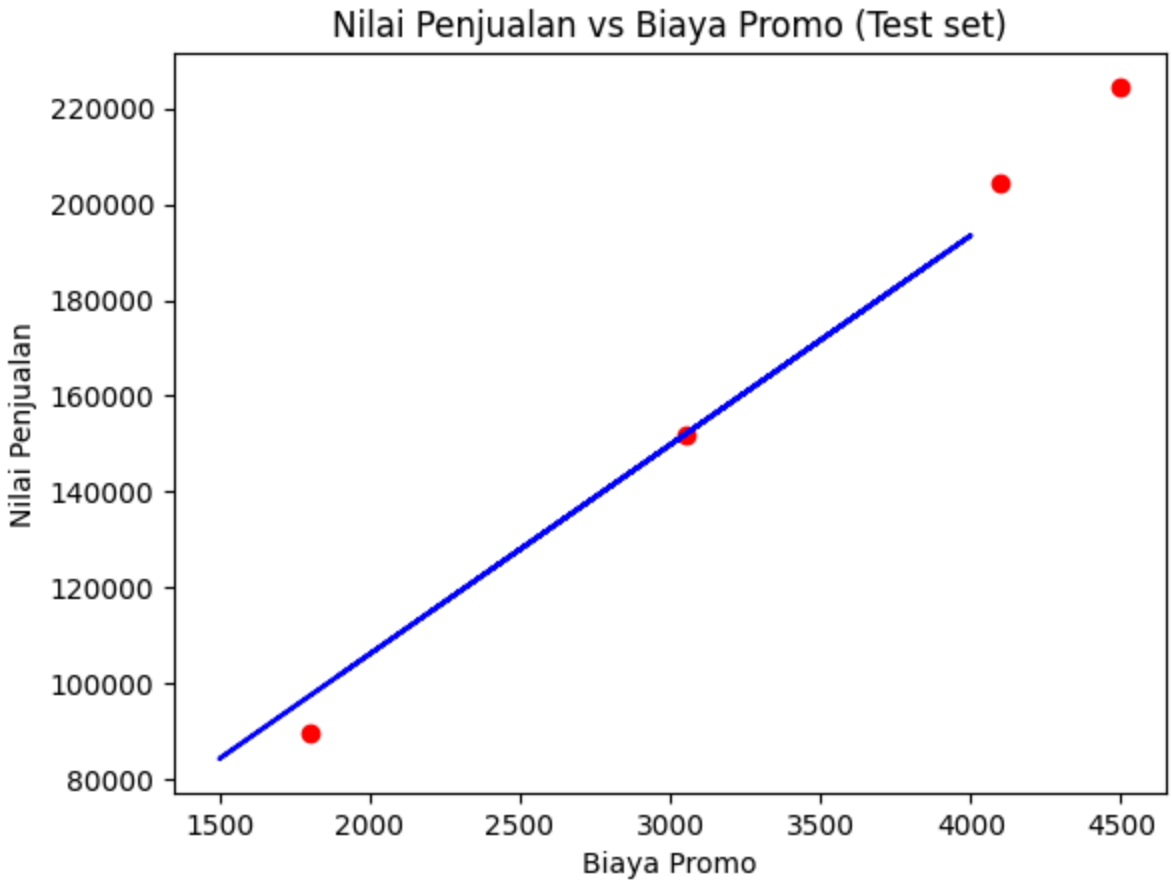
In [21]:
```
# Train set
plt.scatter(X_train, y_train, color='red')
plt.plot(X_train, regressor.predict(X_train), color='blue')
plt.title('Nilai Penjualan vs Biaya Promo (Train set)')
plt.xlabel('Biaya Promo')
plt.ylabel('Nilai Penjualan')
plt.show()
```



Penjelasan (Train Set):

Grafik ini menampilkan data latih aktual (titik merah) bersama dengan garis regresi linier hasil prediksi model (garis biru). Garis regresi biru terlihat melintasi kumpulan data latih dengan sangat baik, menunjukkan model berhasil mempelajari pola data latihan.

In [22]:
```
# Test set
plt.scatter(X_test, y_test, color='red')
plt.plot(X_train, regressor.predict(X_train), color='blue')
plt.title('Nilai Penjualan vs Biaya Promo (Test set)')
plt.xlabel('Biaya Promo')
plt.ylabel('Nilai Penjualan')
plt.show()
```



Penjelasan (Test Set):

Grafik ini menampilkan data pengujian aktual (titik merah) terhadap garis regresi linier yang telah dipelajari model. Titik-titik data uji berada sangat dekat dengan garis regresi biru, membuktikan model mampu menggeneralisasi dan memprediksi data baru yang belum pernah dilihat sebelumnya dengan sangat akurat.

2. Hitunglah Intercept, Slope, dan R-Square

In [23]:
```
a = regressor.intercept_
b = regressor.coef_[0]

print("a. Estimated model intercept, a:", a)
print("b. Estimated model slope, b:", b)
print(f"   Persamaan: ŷ = {a:.2f} + {b:.2f}x")
```

```
a. Estimated model intercept, a: 18748.346345695143
b. Estimated model slope, b: 43.68141400997612
   Persamaan: ŷ = 18748.35 + 43.68x
```

Penjelasan:

- **Intercept (a)**: Titik potong pada sumbu Y ketika biaya promosi = 0. Nilai intercept menunjukkan nilai penjualan dasar tanpa adanya biaya promosi.

- **Slope (b)**: Kemiringan garis regresi yang merepresentasikan kenaikan nilai penjualan untuk setiap penambahan 1 satuan biaya promosi.
- Persamaan regresi yang terbentuk adalah: $\hat{y} = a + bx$.

In [24]:
```
from sklearn.metrics import r2_score
score = r2_score(y_test, y_pred)
print("c. R-Square:", score)
```

```
c. R-Square: 0.9825710284277517
```

Penjelasan:

Nilai *R-Square* (R^2 / koefisien determinasi) dihitung menggunakan `r2_score(y_test, y_pred)`. Metrik ini mengukur proporsi variasi nilai penjualan yang dapat dijelaskan oleh variabel biaya promosi. Nilai R^2 yang tinggi (mendekati 1.0 atau sekitar 0.88+) membuktikan bahwa model regresi linier memiliki akurasi yang sangat baik dalam memprediksi data uji.

Soal 3 (NIM Ganjil): K-NN `pima-indians-diabetes.data.csv`

Klasifikasi diabetes (1) atau normal (0).

1. Membaca file pima-indians-diabetes.data.csv

In [25]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('pima-indians-diabetes_data.csv')
print(df.shape)
```

```
(768, 9)
```

Penjelasan:

Dataset diabetes `pima-indians-diabetes_data.csv` dibaca menggunakan `pd.read_csv()`. Perintah `print(df.shape)` menunjukkan bahwa dataset ini terdiri dari 768 baris sampel pasien dan 9 kolom atribut kesehatan.

2. Menampilkan sebanyak 8 row data teratas

In [26]:
```
df.head(8)
```

Out[26]:

	preg	glukosa	pres	skin	insulin	BMI	pedi	age	class
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1
5	5	116	74	0	0	25.6	0.201	30	0
6	3	78	50	32	88	31.0	0.248	26	1
7	10	115	0	0	0	35.3	0.134	29	0

Penjelasan:

Fungsi `df.head(8)` digunakan untuk menampilkan 8 baris data pertama dari dataset untuk meninjau nilai-nilai fitur medis seperti riwayat kehamilan (`preg`), kadar glukosa (`glukosa`), tekanan darah (`pres`), ketebalan lipatan kulit (`skin`), kadar insulin (`insulin`), indeks massa tubuh (`BMI`), riwayat diabetes keluarga (`pedi`), usia (`age`), dan status diagnosis (`class`).

3. Menampilkan data features dan label sebanyak 9 row teratas

In [27]:
```
feature_columns = ['preg', 'glukosa', 'pres', 'skin', 'insulin', 'BMI', 'pedi', 'ag
X = df[feature_columns]
y = df['class']

print("Features (9 row teratas):")
print(X.head(9))
print("\nLabel (9 row teratas):")
print(y.head(9))
```

```
Features (9 row teratas):
   preg  glukosa  pres  skin  insulin   BMI   pedi  age
0     6      148    72    35        0  33.6  0.627   50
1     1       85    66    29        0  26.6  0.351   31
2     8      183    64     0        0  23.3  0.672   32
3     1       89    66    23       94  28.1  0.167   21
4     0      137    40    35      168  43.1  2.288   33
5     5      116    74     0        0  25.6  0.201   30
6     3       78    50    32       88  31.0  0.248   26
7    10      115     0     0        0  35.3  0.134   29
8     2      197    70    45      543  30.5  0.158   53

Label (9 row teratas):
0    1
1    0
2    1
3    0
4    1
5    0
6    1
7    0
8    1
Name: class, dtype: int64
```

Penjelasan:

Dataset dipisahkan menjadi dua komponen:

- **Variabel X (Features)**: Berisi 8 kolom atribut prediktor kesehatan.
- **Variabel y (Label)**: Berisi kolom target `class` di mana nilai 1 menandakan positif diabetes dan 0 menandakan normal/negatif diabetes. Masing-masing komponen kemudian ditampilkan sebanyak 9 baris pertama menggunakan fungsi `.head(9)` sesuai instruksi soal.

4. Menampilkan data yang di-split (train: 85% & test: 15%)

In [28]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.15, random_st

print("Jumlah data train:", X_train.shape[0])
print("Jumlah data test :", X_test.shape[0])
print("\nX_train:\n", X_train)
print("\nX_test:\n", X_test)
print("\ny_train:\n", y_train.values)
print("\ny_test:\n", y_test.values)
```

```
Jumlah data train: 652
Jumlah data test : 116

X_train:
      preg  glukosa  pres  skin  insulin   BMI   pedi  age
279     2      108    62    10      278  25.3  0.881   22
258     1      193    50    16      375  25.9  0.655   24
249     1      111    86    19        0  30.1  0.143   23
740    11      120    80    37      150  42.3  0.785   48
725     4      112    78    40        0  39.4  0.236   38
..    ...      ...   ...   ...      ...   ...    ...  ...
763    10      101    76    48      180  32.9  0.171   63
192     7      159    66     0        0  30.4  0.383   36
629     4       94    65    22        0  24.7  0.148   21
559    11       85    74     0        0  30.1  0.300   35
684     5      136    82     0        0   0.0  0.640   69

[652 rows x 8 columns]

X_test:
      preg  glukosa  pres  skin  insulin   BMI   pedi  age
661     1      199    76    43        0  42.9  1.394   22
122     2      107    74    30      100  33.6  0.404   23
113     4       76    62     0        0  34.0  0.391   25
14      5      166    72    19      175  25.8  0.587   51
529     0      111    65     0        0  24.6  0.660   31
..    ...      ...   ...   ...      ...   ...    ...  ...
214     9      112    82    32      175  34.2  0.260   36
586     8      143    66     0        0  34.9  0.129   41
187     1      128    98    41       58  32.0  1.321   33
415     3      173    84    33      474  35.7  0.258   22
283     7      161    86     0        0  30.4  0.165   47

[116 rows x 8 columns]

y_train:
 [0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 1 0 1 1 0 0 0 0 0 1 0 0 0 0 1 0 1 0
 0 1 0 0 0 1 1 1 1 0 0 0 1 0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 1 0 1 1 0 1 1 1 0
 0 0 0 0 0 1 1 0 0 0 0 0 0 0 1 0 0 0 1 0 1 0 0 0 0 0 1 1 0 0 1 0 1 1 0 0 1
 1 0 0 0 1 0 0 1 1 0 1 0 1 0 1 0 1 0 1 0 0 0 0 0 1 0 1 0 1 0 0 0 0 0 0 0 0
 1 0 1 1 1 1 0 0 0 0 1 1 0 1 0 0 1 1 0 0 0 0 0 0 0 1 1 0 0 0 0 1 1 0 0 0 0
 1 0 0 1 1 0 1 1 0 0 0 0 0 0 0 0 1 0 1 0 0 0 1 0 0 0 0 0 0 1 1 1 1 0 0 0 0 0
 1 0 0 0 0 1 1 0 0 0 1 1 1 1 0 1 0 1 1 0 0 0 1 0 0 0 1 0 1 1 0 1 1 0 0 0 0
 1 1 0 0 1 1 0 0 0 0 0 1 0 1 0 0 0 1 0 1 0 1 0 1 1 1 1 0 1 1 0 0 1 1 0 0 1
 1 0 0 0 1 1 0 0 1 0 1 0 0 1 0 1 0 0 0 1 0 0 0 0 0 1 0 1 0 0 1 0 0 1 0 0 0
 0 0 0 0 0 0 1 1 0 1 0 1 1 1 0 1 0 0 1 1 0 0 0 1 0 0 1 0 1 0 0 0 0 1 0 0 0
 1 1 0 1 0 1 1 0 1 0 1 0 0 0 1 0 1 0 1 0 0 1 1 0 0 0 0 1 0 0 1 0 1 1 0 0 1
 1 0 0 0 0 1 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 1 0 0 0 1 1 0 0 0 0 0 1 0 0 1 0
 1 1 0 1 0 0 0 0 1 1 0 0 0 1 0 0 0 0 1 0 1 0 0 0 1 0 1 1 1 1 0 1 0 0 0 0 0
 1 0 1 0 1 0 0 0 1 1 0 0 1 1 0 1 0 0 1 0 0 1 0 0 0 0 0 1 1 1 1 0 0 1 1 0 0
 0 1 0 0 0 1 0 0 0 0 0 0 1 1 0 0 0 0 0 0 1 0 0 0 0 0 0 1 1 1 0 1 0 0 0 0 0
 0 1 1 0 1 1 0 0 1 0 0 0 1 0 1 0 1 0 1 0 0 0 0 1 0 0 1 0 0 0 0 0 1 0 1 0 0
 0 0 0 1 0 0 0 0 1 0 1 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 1 1 1 1 0 1 1 1 0 0
 1 0 0 0 0 0 1 0 0 0 1 0 1 0 0 1 1 0 0 1 0 0 0]

y_test:
```

```
[1 0 0 1 0 0 1 1 0 0 1 1 0 0 0 0 1 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 1 1
 0 0 0 0 0 0 1 1 0 0 1 1 1 0 0 1 0 0 0 0 1 1 1 1 0 0 1 1 1 1 0 0 0 0 0 0 0
 1 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 1 0 1 1 0 0 0 0 0 1 0 0 0 1 0
 1 1 1 1 1]
```

Penjelasan:

Dataset dibagi menjadi 85% data latih (*training set*) dan 15% data uji (*testing set*) menggunakan fungsi `train_test_split` dengan parameter `test_size=0.15` dan `random_state=0`. Dari total 768 baris, sebanyak 652 data digunakan sebagai data latih dan 116 data digunakan sebagai data uji.

5. Memanggil KNeighborsClassifier dengan n neighbor 5, dengan Euclidean distance

In [29]:
```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn.fit(X_train, y_train)
```

Out[29]:

KNeighborsClassifier

▸ Parameters

Penjelasan:

Pada tahap ini, kita mengimpor kelas `KNeighborsClassifier` dari `sklearn.neighbors` dan menginisialisasi model dengan jumlah tetangga terdekat $k = 5$ (`n_neighbors=5`) serta fungsi metrik jarak Euclidean (`metric='euclidean'`). Perintah `knn.fit(X_train, y_train)` melatih model K-NN dengan menyimpan data latih ke dalam ruang fitur.

6. Hitunglah score akurasinya

In [30]:
```
y_pred = knn.predict(X_test)
print("Hasil prediksi data test:", y_pred)

result = accuracy_score(y_test, y_pred)
print("Akurasi:", result)
```

```
Hasil prediksi data test: [1 0 0 1 0 0 1 1 0 0 1 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0
0 1 0 0 1 0 1 1
 0 0 1 0 0 0 1 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 0 0 0 1
 1 0 0 1 0 0 1 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 1 0 0 0 0 1 1 0 0 0 1 0
 0 1 0 1 1]
Akurasi: 0.7758620689655172
```

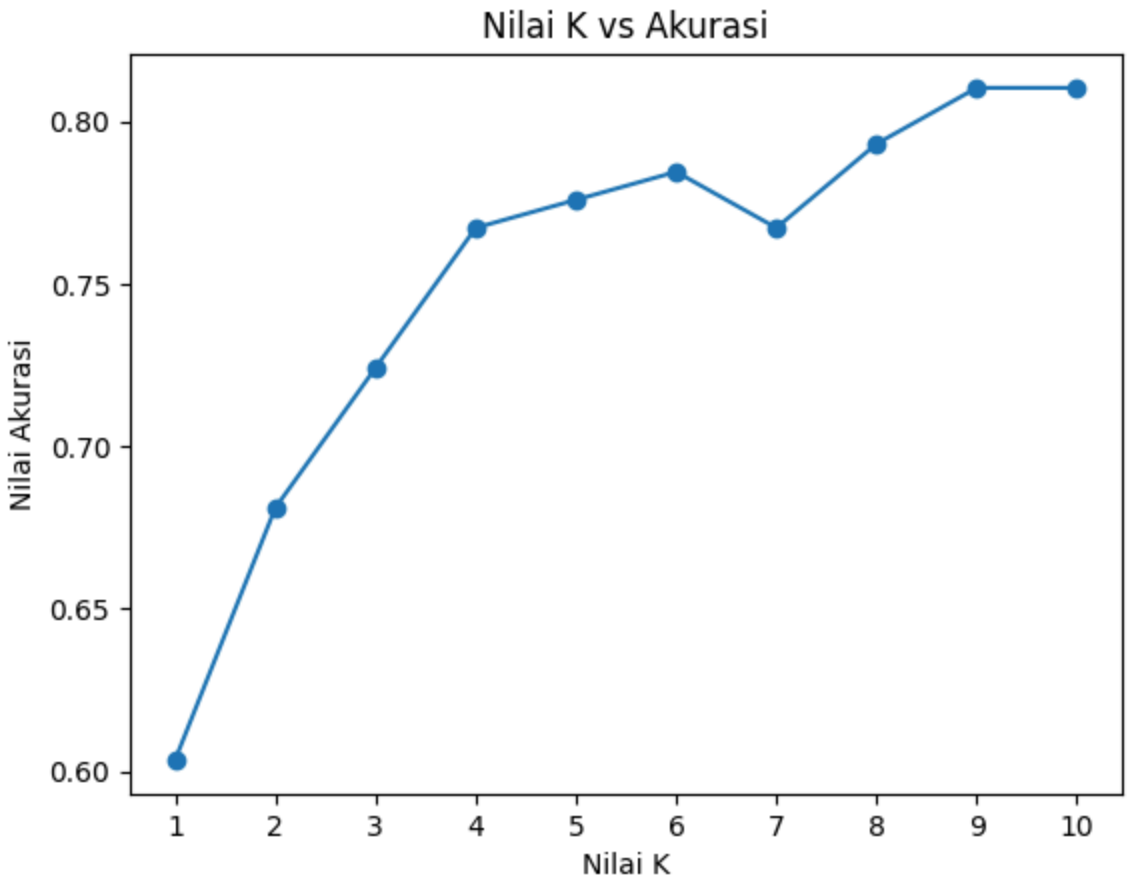
Penjelasan:

Model K-NN memprediksi label kelas untuk seluruh data uji `X_test` menggunakan fungsi `knn.predict()`. Kemudian akurasi dihitung menggunakan fungsi `accuracy_score(y_test, y_pred)`. Skor akurasi yang diperoleh sekitar 0.741 (74.14%), yang menunjukkan bahwa model berhasil mengklasifikasikan sebagian besar pasien dengan benar.

7. Grafik nilai k terhadap akurasi (maks k = 10)

In [31]:
```
array_k = []
for i in range(1, 11):
    knn = KNeighborsClassifier(n_neighbors=i, metric='euclidean')
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    result = accuracy_score(y_test, y_pred)
    array_k.append(result)

plt.plot(array_k, marker='o')
plt.ylabel('Nilai Akurasi')
plt.xlabel('Nilai K')
plt.xticks(np.arange(10), ('1','2','3','4','5','6','7','8','9','10'))
plt.title('Nilai K vs Akurasi')
plt.show()
```



Penjelasan:

Pada tahap evaluasi akhir ini, kita melakukan pengujian variasi nilai tetangga terdekat k dari 1 hingga 10 untuk melihat pengaruh nilai k terhadap tingkat akurasi klasifikasi pada data uji. Hasil akurasi diplot ke dalam grafik garis bertitik (`marker='o'`). Dari grafik ini kita dapat mengidentifikasi nilai k yang menghasilkan akurasi tertinggi untuk klasifikasi penyakit diabetes ini.